

Chapter 2

How Spring Boot Starts

A step-by-step walkthrough of the Spring Boot startup sequence — from `main()` execution to a running, request-ready application.

2.1 Introduction

Before our application can register users, generate JWTs, or communicate with the database, it must first start successfully.

When we click the **Run** button in our IDE, a series of operations take place behind the scenes. Spring Boot initializes the application, creates all the required objects (beans), starts the embedded Tomcat server, and prepares the application to receive HTTP requests.

Understanding this startup process helps us understand how Spring Boot manages objects, dependency injection, and request processing.

2.2 Starting the Application

The application starts from the following class, called the **entry point** of the application:

```
@SpringBootApplication
public class FoodBridgeApplication {

    public static void main(String[] args) {
        SpringApplication.run(FoodBridgeApplication.class, args);
    }

}
```

Whenever we execute this class, Java calls the `main()` method first.

The Nine-Step Startup Sequence

1 JVM Executes main()

```
public static void main(String[] args)
```

WHAT HAPPENS

The Java Virtual Machine (JVM) starts the program and executes the `main()` method — the first method executed in every Java application.

WHY

The JVM always needs a starting point. The `main()` method acts as the entry point from where the entire application begins execution. Without this method, Java would not know where to start the program.

2 SpringApplication.run()

```
SpringApplication.run(FoodBridgeApplication.class, args);
```

WHAT HAPPENS

This single line starts the entire Spring Boot framework. Internally it performs hundreds of operations, including:

- Creates the Spring IoC Container
- Performs Component Scanning
- Creates all Spring Beans
- Configures the application
- Starts the embedded Tomcat server
- Makes the application ready to receive requests

WHY

Instead of writing all these initialization steps manually, Spring Boot automates the entire startup process — one of its biggest advantages.

3 Spring Creates the IoC Container

WHAT HAPPENS

Spring creates an object called the **IoC (Inversion of Control) Container**, responsible for managing every object required by the application — for example `UserService`, `JwtService`, `JwtFilter`, `UserRepository`, `BCryptPasswordEncoder`, and `SecurityConfig`. Instead of us creating these objects with the `new` keyword, Spring creates and manages them.

WHY

In an application with hundreds of classes, creating every object manually would make it difficult to maintain. The IoC Container centralizes object creation and lifecycle management, keeping the application loosely coupled and easier to maintain.

4 Component Scanning

WHAT HAPPENS

Spring scans the project's packages looking for annotations such as `@Component`, `@Service`, `@Repository`, `@Controller`, and `@Configuration`. Whenever it finds one, it understands that the class should be managed by the IoC Container.

WHY

Spring must know which classes should become managed objects. Without component scanning, Spring would never discover classes like `JwtFilter` or `UserService`.

5 Bean Creation

WHAT HAPPENS

For every discovered component, Spring creates exactly one object by default. These objects are called **Beans**:

```
UserService    → Bean Created  
JwtService     → Bean Created  
UserRepository → Bean Created
```

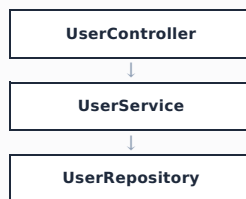
WHY

Beans allow Spring to reuse the same object throughout the application. Instead of creating multiple copies of the same class, Spring provides the existing bean whenever it is needed — improving performance and simplifying dependency management.

6 Dependency Injection

WHAT HAPPENS

After creating all beans, Spring connects them together. It notices that `UserController` requires a `UserService`, and automatically injects that bean through the constructor:



Similarly, `UserService` receives `UserRepository`, `PasswordEncoder`, and `JwtService` without creating them manually.

WHY

Dependency Injection removes the responsibility of creating dependent objects from the programmer. Instead of writing `UserService service = new UserService(...)`, Spring automatically supplies the required objects — keeping the code clean, reusable, and easy to test.

7 Configuration Classes

WHAT HAPPENS

Spring processes classes annotated with `@Configuration`. Inside these classes, methods marked with `@Bean` are executed:

```
@Bean  
public BCryptPasswordEncoder hashPassword() {  
    return new BCryptPasswordEncoder();  
}
```

Spring stores the returned object as a bean.

WHY

Some classes belong to external libraries and cannot be annotated with `@Component`. Using `@Bean` allows us to register such objects manually.

8 Embedded Tomcat Starts

WHAT HAPPENS

Spring Boot automatically starts an embedded Tomcat server, which begins listening for incoming HTTP requests on the configured port (default: **8080**).

WHY

Traditional Java web applications required developers to install Tomcat separately. Spring Boot embeds Tomcat inside the application, making deployment much easier.

9 Application is Ready

WHAT HAPPENS

Once all beans are created, dependencies are injected, configurations are applied, and Tomcat starts successfully, Spring Boot prints:

```
Started FoodBridgeApplication in ...
```

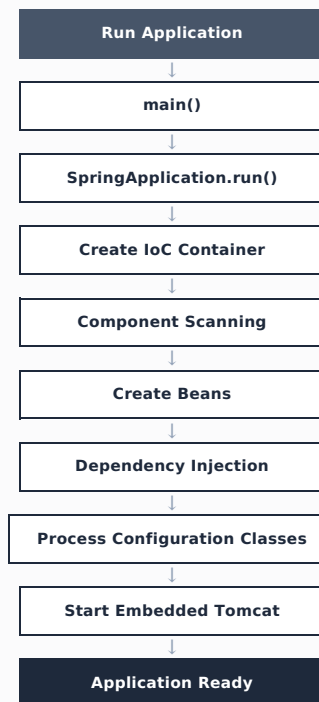
The application is now ready to receive client requests.

WHY

At this point, every required object already exists. Whenever a request arrives, Spring can immediately process it without creating objects again.

Complete Startup Flow

SPRING BOOT STARTUP SEQUENCE



Concepts Learned

JVM	main() Method	SpringApplication.run()	IoC Container	Dependency Injection	Bean	Component Scanning	@Component
@Service	@Repository	@Controller	@Configuration	@Bean	Embedded Tomcat		

CHAPTER SUMMARY

When we run the application, the JVM starts the `main()` method, which calls `SpringApplication.run()` . Spring Boot then creates the IoC Container, scans the project for components, creates and wires all beans using Dependency Injection, processes configuration classes, starts the embedded Tomcat server, and prepares the application to receive HTTP requests.

This startup sequence forms the foundation of every Spring Boot application and explains why our controllers, services, repositories, filters, and configuration classes are available without manually creating them.